

OmniFetch Engine: Master Blueprint

High-Performance macOS Download Accelerator (.NET MAUI & Manifest V3 Architecture)

PRODUCTION ENGINEERING DIRECTIVE

1. System Architecture Overview

OmniFetch is an enterprise-grade download accelerator designed specifically for macOS, replicating and exceeding the architectural capabilities of Windows Internet Download Manager (IDM). It decouples download detection, inter-process communication, and multi-threaded stream orchestration into a resilient 3-tier architecture.

Layer	Technology	Core Responsibilities
Browser Scout	Chrome MV3 Extension	Sniffs HTTP traffic, intercepts download requests, gathers authentication cookies, captures browser headers, and cancels default browser file operations.
Native IPC Bridge	C# Native AOT Console	Headless standard I/O daemon registered with macOS Native Messaging Hosts. Translates Chrome JSON payloads to local Unix Domain Sockets.
Core Application	.NET MAUI (Mac Catalyst)	Main desktop GUI, SQLite queue database, zero-allocation network engine, lock-free disk I/O writer, and token bucket bandwidth throttling.

2. The High-Speed Multi-Thread Engine

IDM's speed advantage stems from multi-connection HTTP range requests combined with dynamic file segmentation. Conventional downloaders write temporary files to disk and stitch them together at the end—a process that doubles disk wear and freezes the system during the concatenation phase. OmniFetch eliminates this bottleneck entirely.

2.1 Server Capability Probe

Before launching download threads, OmniFetch sends an initial HTTP HEAD request to probe the target server:

- **Accept-Ranges:** Validates whether the server supports bytes range requests. If absent, the engine defaults to a single-stream fallback download.
- **Content-Length:** Captures the exact file size in bytes.
- **ETag / Last-Modified:** Persisted in the database to verify file consistency when resuming paused or broken connections.

2.2 Zero-Allocation Lock-Free Disk I/O

Upon discovering the file size, OmniFetch pre-allocates the complete target file on disk using native file system calls. It uses .NET's high-performance RandomAccess.WriteAsync API via SafeFileHandle, allowing concurrent

worker threads to write their downloaded byte buffers directly into their respective file positions without locks or mutex contention.

Zero-Bug Directive: Memory Leaks & Garbage Collection

Never read complete stream chunks into managed `byte[]` arrays. Always stream incoming data using `HttpCompletionOption.ResponseHeadersRead`. Rent reusable 81,920-byte buffers from `ArrayPool<byte>.Shared`, copy into `Memory<byte>`, flush to disk via `RandomAccess.WriteAsync`, and immediately return the buffer to the pool in a `finally` block.

```
// High-Throughput Chunk Writer Routine
public async Task DownloadSegmentAsync(SafeFileHandle fileHandle, string url, long startOffset,
long endOffset, CancellationToken ct)
{
    using var request = new HttpRequestMessage(HttpMethod.Get, url);
    request.Headers.Range = new RangeHeaderValue(startOffset, endOffset);

    using var response = await _httpClient.SendAsync(request,
HttpCompletionOption.ResponseHeadersRead, ct);
    response.EnsureSuccessStatusCode();

    await using var responseStream = await response.Content.ReadAsStreamAsync(ct);
    byte[] buffer = ArrayPool<byte>.Shared.Rent(81920);

    try
    {
        long currentFileOffset = startOffset;
        int bytesRead;
        while ((bytesRead = await responseStream.ReadAsync(buffer.AsMemory(0, buffer.Length),
ct)) > 0)
        {
            await RandomAccess.WriteAsync(fileHandle, buffer.AsMemory(0, bytesRead),
currentFileOffset, ct);
            currentFileOffset += bytesRead;
            _segmentTracker.UpdateProgress(startOffset, currentFileOffset);
        }
    }
    finally
    {
        ArrayPool<byte>.Shared.Return(buffer);
    }
}
```

2.3 Dynamic Resegmentation Algorithm

Network latency varies dynamically across parallel TCP connections. If 8 threads are downloading an 800 MB file (100 MB each) and Thread 1 finishes while Thread 8 is stalled at 20%, OmniFetch activates dynamic resegmentation:

1. The segment manager locates the active segment with the largest remaining byte delta: `remainingBytes = EndByte - CurrentByte`.

2. If `remainingBytes > (MinSplitThreshold = 5 MB)`, calculate midpoint: `splitPoint = CurrentByte + (remainingBytes / 2)`.
3. Atomically clamp the original segment's `EndByte` to `splitPoint - 1`.
4. Spawn an idle worker task assigned to the new range: `[splitPoint, originalEndByte]`.

3. State Persistence & Resumption Mechanics

Reliability requires instantaneous recovery from network drops, system sleep, or app terminations. OmniFetch utilizes Entity Framework Core backed by a local SQLite database.

3.1 Database Schema

```
// SQLite Entity Models
public enum DownloadStatus { Queued, Downloading, Paused, Completed, Failed, Expired }

public class DownloadJob
{
    public Guid Id { get; set; } = Guid.NewGuid();
    public string Url { get; set; } = string.Empty;
    public string SavePath { get; set; } = string.Empty;
    public long TotalBytes { get; set; }
    public string? ETag { get; set; }
    public string? LastModified { get; set; }
    public string? Cookies { get; set; }
    public string? UserAgent { get; set; }
    public string? Referrer { get; set; }
    public DownloadStatus Status { get; set; }
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
    public List<DownloadSegment> Segments { get; set; } = new();
}

public class DownloadSegment
{
    public Guid Id { get; set; } = Guid.NewGuid();
    public Guid JobId { get; set; }
    public long StartByte { get; set; }
    public long EndByte { get; set; }
    public long CurrentByte { get; set; }
    public bool IsCompleted => CurrentByte >= EndByte;
}
```

3.2 Flush Frequency & Transaction Decoupling

Network threads must never block on SQLite database transactions. Segment progress is tracked in thread-safe memory accumulators and committed in a batched asynchronous background task every 2,000 milliseconds.

3.3 "Refresh Download Address" Workflow

When downloading from temporary URL hosts (e.g., file-sharing sites, expiring signed S3 URLs), an expired link returns HTTP **403 Forbidden** or **410 Gone** upon resume. OmniFetch replicates IDM's refresh flow:

- The download job pauses and is flagged as **Status = Expired**.
- OmniFetch signals the browser extension or opens the original Referrer webpage in Chrome.
- When the user clicks download on the webpage, the Chrome Extension detects the matching file signature and forwards the fresh URL and new authentication cookies.

- OmniFetch updates the database records with the new URL and resumes all uncompleted segments without discarding previously written bytes.

4. macOS Platform Engineering (Mac Catalyst)

4.1 App Nap & Sleep Prevention

macOS aggressively throttles background network and CPU activity for minimized applications via "App Nap". OmniFetch must instruct `NSProcessInfo` that an active transfer is latency-critical.

```
#if MACCATALYST
using Foundation;

public class MacActivityLock : IDisposable
{
    private NSObject? _activity;

    public void PreventAppNap(string reason)
    {
        var options = NSActivityOptions.UserInitiated | NSActivityOptions.LatencyCritical;
        _activity = NSProcessInfo.ProcessInfo.BeginActivity(options, reason);
    }

    public void Dispose()
    {
        if (_activity != null)
        {
            NSProcessInfo.ProcessInfo.EndActivity(_activity);
            _activity = null;
        }
    }
}
#endif
```

4.2 macOS App Sandboxing & Entitlements

To allow saving files anywhere in the user's filesystem, configure `Entitlements.plist`:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/
PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
  <key>com.apple.security.app-sandbox</key>
  <true/>
  <key>com.apple.security.network.client</key>
  <true/>
  <key>com.apple.security.files.user-selected.read-write</key>
  <true/>
  <key>com.apple.security.files.downloads.read-write</key>
  <true/>
</dict>
</plist>
```

5. The Native Messaging Host & IPC Bridge

Because sandboxed Mac Catalyst applications cannot be launched directly by Chrome via Native Messaging standard I/O, OmniFetch utilizes a native bridge console binary compiled with .NET Native AOT.

5.1 Registration Manifest

Deploy

to:

`~/Library/Application Support/Google/Chrome/NativeMessagingHosts/com.omnifetch.bridge.json`

```
{
  "name": "com.omnifetch.bridge",
  "description": "OmniFetch Native Messaging IPC Bridge",
  "path": "/Applications/OmniFetch.app/Contents/MacOS/OmniFetch.Bridge",
  "type": "stdio",
  "allowed_origins": [
    "chrome-extension://<EXTENSION_ID>/"
  ]
}
```

5.2 Bridge Protocol Implementation

Chrome communicates using 32-bit little-endian integer prefixing followed by UTF-8 JSON. The Bridge reads the message and relays it via a local Unix Domain Socket (or macOS Named Pipe):

```
// OmniFetch.Bridge Native AOT Implementation
public static async Task Main(string[] args)
{
    using var stdin = Console.OpenStandardInput();
    using var stdout = Console.OpenStandardOutput();

    byte[] lengthBuffer = new byte[4];
    int read = await stdin.ReadAsync(lengthBuffer.AsMemory(0, 4));
    if (read < 4) return;

    int messageLength = BitConverter.ToInt32(lengthBuffer, 0);
    byte[] messageBuffer = new byte[messageLength];
    await stdin.ReadExactlyAsync(messageBuffer.AsMemory(0, messageLength));

    string jsonPayload = Encoding.UTF8.GetString(messageBuffer);

    // Relay to MAUI App via Unix Domain Socket
    var socketPath =
Path.Combine(Environment.GetFolderPath(Environment.SpecialFolder.UserProfile),
".omnifetch.sock");
    using var socket = new Socket(AddressFamily.Unix, SocketType.Stream,
ProtocolType.Unspecified);
    await socket.ConnectAsync(new UnixDomainSocketEndPoint(socketPath));
    await socket.SendAsync(Encoding.UTF8.GetBytes(jsonPayload));

    // Acknowledge Chrome
    string ack = "{\"status\":\"forwarded\"}";
    byte[] ackBytes = Encoding.UTF8.GetBytes(ack);
    await stdout.WriteAsync(BitConverter.GetBytes(ackBytes.Length));
    await stdout.WriteAsync(ackBytes);
}
```

6. Chrome Extension (Manifest V3) Specification

The Chrome extension runs in the background and intercepts file downloads before the browser initiates them.

6.1 `manifest.json`

```
{
  "manifest_version": 3,
  "name": "OmniFetch Integration",
  "version": "1.0.0",
  "description": "Seamlessly routes downloads to OmniFetch for macOS.",
  "permissions": [
    "downloads",
    "cookies",
    "nativeMessaging",
    "webRequest",
    "declarativeNetRequest"
  ],
  "host_permissions": [
    "<all_urls>"
  ],
  "background": {
    "service_worker": "background.js"
  },
  "icons": {
    "128": "icon128.png"
  }
}
```

6.2 background.js

```
const NATIVE_HOST = "com.omnifetch.bridge";

chrome.downloads.onDeterminingFilename.addListener((downloadItem, suggest) => {
  // 1. Instantly cancel native download
  chrome.downloads.cancel(downloadItem.id);

  const targetUrl = downloadItem.finalUrl || downloadItem.url;
  const urlObj = new URL(targetUrl);

  // 2. Extract authentication cookies for domain
  chrome.cookies.getAll({ domain: urlObj.hostname }, (cookies) => {
    const cookieHeader = cookies.map(c => `${c.name}=${c.value}`).join("; ");

    const payload = {
      url: targetUrl,
      referrer: downloadItem.referrer,
      cookies: cookieHeader,
      userAgent: navigator.userAgent,
      mimeType: downloadItem.mimeType,
      fileSize: downloadItem.fileSize,
      suggestedFileName: downloadItem.suggestedFileName
    };

    // 3. Dispatch to macOS Native Host
    chrome.runtime.sendMessage(NATIVE_HOST, payload, (response) => {
      if (chrome.runtime.lastError) {
        console.error("IPC Bridge Error:", chrome.runtime.lastError.message);
      }
    });
  });
});
```

7. HLS & DASH Video Assembly Pipeline

Modern video streaming sites do not serve monolithic `.mp4` files. Instead, they serve HLS (HTTP Live Streaming) `.m3u8` playlists or DASH `.mpd` manifests pointing to segmented Transport Stream (`.ts`) or fragmented MP4 (`.m4s`) files.

1. **Manifest Parsing:** Parse the master playlist, select the highest available bitrate stream, and index all chunk URIs.
2. **Parallel Chunk Fetching:** Download chunks in parallel batches using the core engine's connection pool, naming them sequentially (e.g., `segment_0001.ts`).
3. **Bundled FFmpeg Assembly:** Bundle a signed macOS ARM64/x64 FFmpeg static binary inside the application bundle. Concatenate chunks losslessly:

```
ffmpeg -f concat -safe 0 -i chunks.txt -c copy -bsf:a aac_adtstoasc final_output.mp4
```

8. UI Responsiveness & Strict Accessibility

8.1 Reactive UI Throttling

During Gigabit downloads, network threads emit thousands of progress updates per second. Binding these directly to XAML visual elements will overwhelm the Mac Catalyst UI thread and cause application hangs.

UI Directive: Event Throttling

Use Reactive Extensions (Rx) `Observable.Sample(TimeSpan.FromMilliseconds(250))` or a dedicated `System.Threading.Timer` to batch and emit UI telemetry at a maximum frequency of 4 Hz (every 250ms).

8.2 Accessibility (VoiceOver & Keyboard Navigation)

Every UI component in the .NET MAUI view must comply with accessibility guidelines:

- Assign explicit `AutomationProperties.Name` and `SemanticProperties.Description` to all control buttons (Pause, Resume, Cancel, Limit Speed).
- Ensure all progress indicators update their `SemanticProperties.Hint` with current percentage and download speed so VoiceOver users receive coherent updates.
- Support full keyboard accessibility: Provide logical tab indexes and keybindings (`Space` to pause/resume, `Cmd+Backspace` to cancel).

9. AI Agent Step-by-Step Prompting Sequence

Instruct your AI agent (Cursor, Claude, or ChatGPT) to build OmniFetch sequentially. **Do not execute all phases at once.** Follow this phased roadmap:

Phase	Component	Instructions for AI Agent
Phase 1	Core Engine Class Library	"Build the .NET Class Library for multi-stream downloading using SocketsHttpHandler, ArrayPool<byte>, and lock-free RandomAccess.WriteAsync. Implement dynamic byte resegmentation."
Phase 2	SQLite State Engine	"Implement Entity Framework Core models (DownloadJob, DownloadSegment) with SQLite. Add asynchronous write-behind progress flushing and resume verification using ETags."
Phase 3	Native AOT IPC Bridge	"Develop the Native AOT C# console daemon for Chrome Native Messaging. Read 32-bit length-prefixed JSON from standard input and write to a macOS Unix Domain Socket."
Phase 4	Chrome Extension	"Create the Manifest V3 extension with downloads, cookies, and nativeMessaging permissions. Intercept downloads and forward complete context to com.omnifetch.bridge."
Phase 5	MAUI Mac Catalyst Shell	"Build the .NET MAUI Mac Catalyst UI using CommunityToolkit.Mvvm. Implement throttled UI updates (250ms), full VoiceOver accessibility properties, and NSProcessInfo App Nap prevention."
Phase 6	HLS/FFmpeg Integration	"Add M3U8 parser support and bundled FFmpeg process invocation for lossless Transport Stream video stitching."